



Reinforcement Learning I: MDP

Course AIAA 4220

Week 9- Lecture 17

Changhao Chen

Assistant Professor

HKUST (GZ)

Recap: Planning for Embodied AI

Definition: Finding a sequence of actions to achieve a goal given constraints and a model of the world.

Analogy: “Navigation GPS for robots.”

Planning vs. Decision-making vs. Control

- Planning: *Strategic* (where to go)
- Control: *Tactical* (how to move)
- Decision-making: *High-level* (what to achieve)



Recap: Planning for Robotics

Popular Approaches

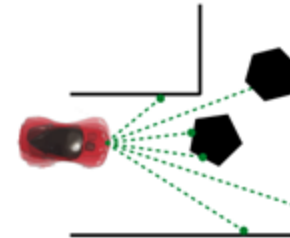
- **Potential fields** [*Rimon, Koditschek, '92*]: create forces on the robot that pull it toward the goal and push it away from obstacles
- **Grid-based planning** [*Stentz, '94*]: discretizes problem into grid and runs a graph-search algorithm (Dijkstra, A*, ...)
- **Combinatorial planning** [*LaValle, '06*]: constructs structures in the configuration space (C-space) that completely capture all information needed for planning
- **Sampling-based planning** [*Kavraki et al, '96; LaValle, Kuffner, '06, etc.*]: uses collision detection algorithms to probe and incrementally search the C-space for a solution

Recap: Robotic Control

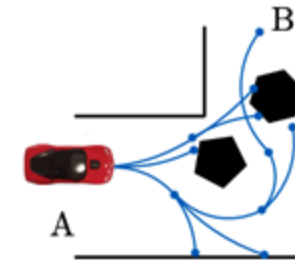
What Is Control?

- Definition: **Ensuring the robot follows planned motion despite disturbances and model errors.**
- Role in embodied AI: Bridge between planning and actuation.

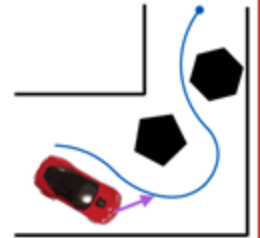
Estimate state



Plan a sequence of motions



Control robot to follow path



- Robot pose known
- Path is given

Recap: PID Controller

Control input $u(t)$ is proportional to **error**, its **accumulation**, and its **rate of change**.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Where:

- $e(t) = r(t) - y(t)$: tracking error
- K_p : proportional gain
- K_i : integral gain
- K_d : derivative gain

Recap: MPC

Model Predictive Control (MPC)

At each time step, **predict future behavior** using a system model and **optimize future control inputs** to minimize a cost function.

$$\min_{u_{0:H}} \sum_{k=0}^H \left[(x_k - x_{\text{goal}})^T Q (x_k - x_{\text{goal}}) + u_k^T R u_k \right]$$

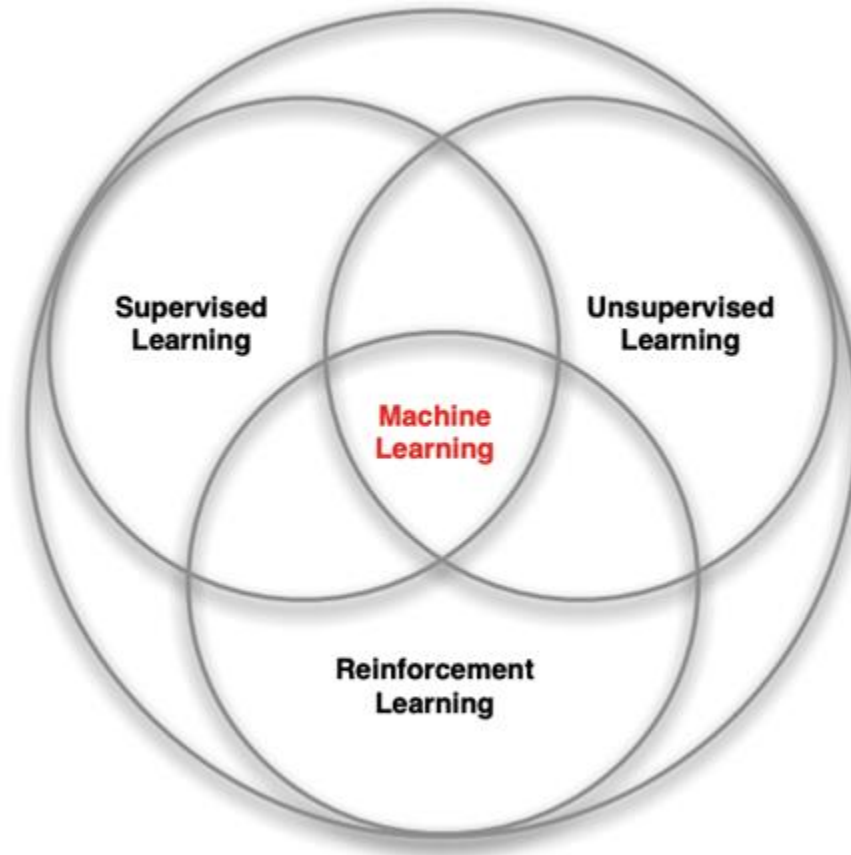
- Q : penalty for state error (tracking error)
- R : penalty for control effort
- Balances **accuracy vs smooth control**

MPC finds control inputs that track target while minimizing effort and respecting constraints.

Introduction to RL

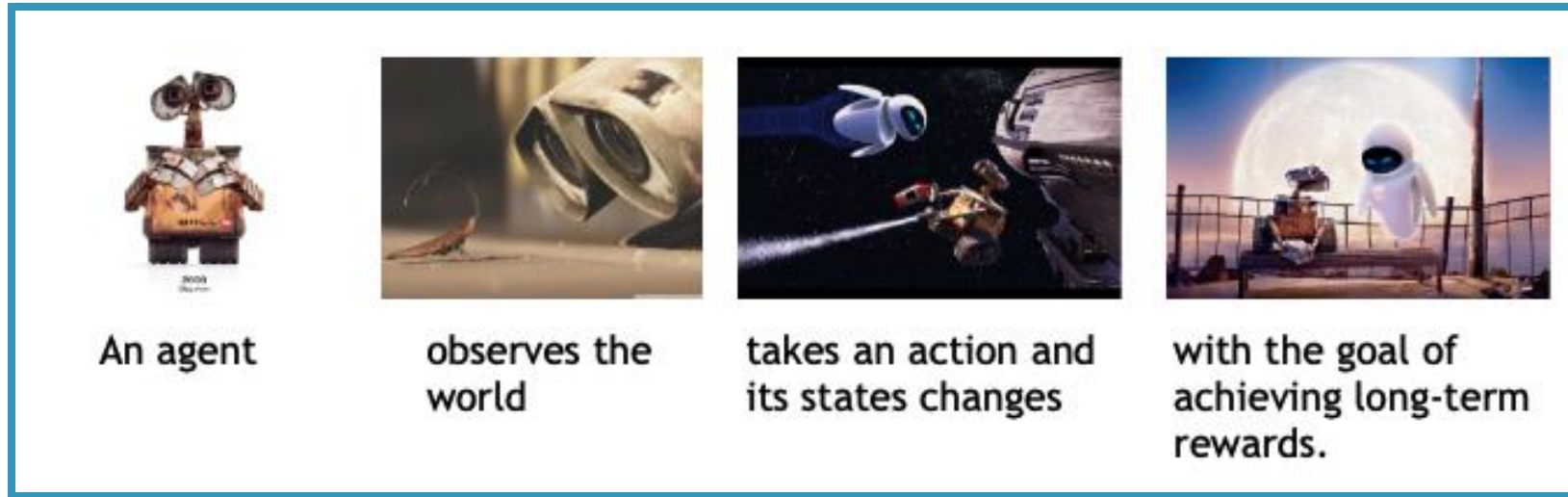
Branches of Machine Learning:

- Supervised learning: labels of desired behavior
- Unsupervised learning: no labels
- **Reinforcement learning: reward signal evaluating the outcome of past actions**



RL Problem

- In RL, we typically focus on **sequential decision making**: an agent chooses a sequence of actions which each affect future possibilities available to the agent.

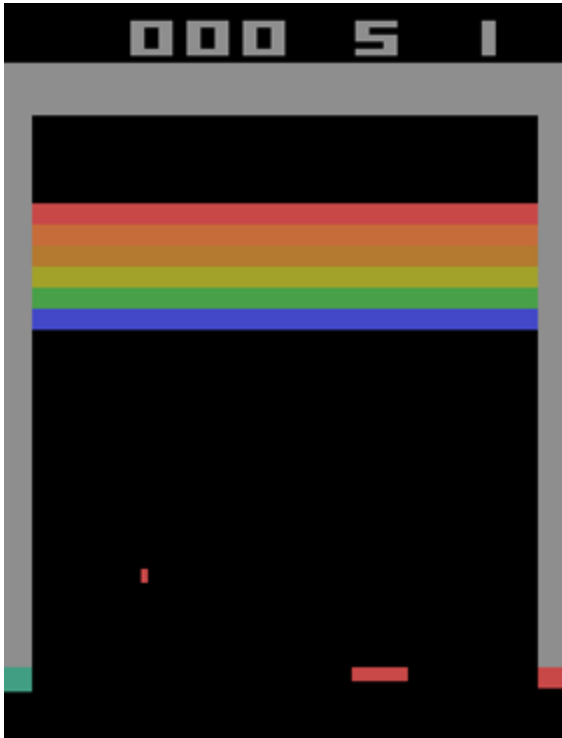


- What makes reinforcement learning different from other machine learning paradigms?
 - There is no supervisor, **only a reward signal**
 - **Feedback is delayed, not instantaneous**
 - That means we need to run the policy network for a number of steps, then do gradient descent
 - Time really matters (sequential, non i.i.d data)
 - Agent's actions affect the subsequent data it receives

Introduction to RL

Examples of Reinforcement Learning

- Fly stunt manoeuvres in a helicopter
- Manage an investment portfolio
- Control a power station
- Make a humanoid robot walk
- Play many different Atari games better than humans



RL Plays Atari games



Make a humanoid robot walk

Rewards

- **A reward R_t is a scalar feedback signal**
 - Indicates how well agent is doing at step t
- The agent's job is to maximise cumulative reward
- Reinforcement learning is based on the **reward** hypothesis
 - **All goals can be described by the maximisation of expected cumulative reward**



Introduction

Examples of Rewards

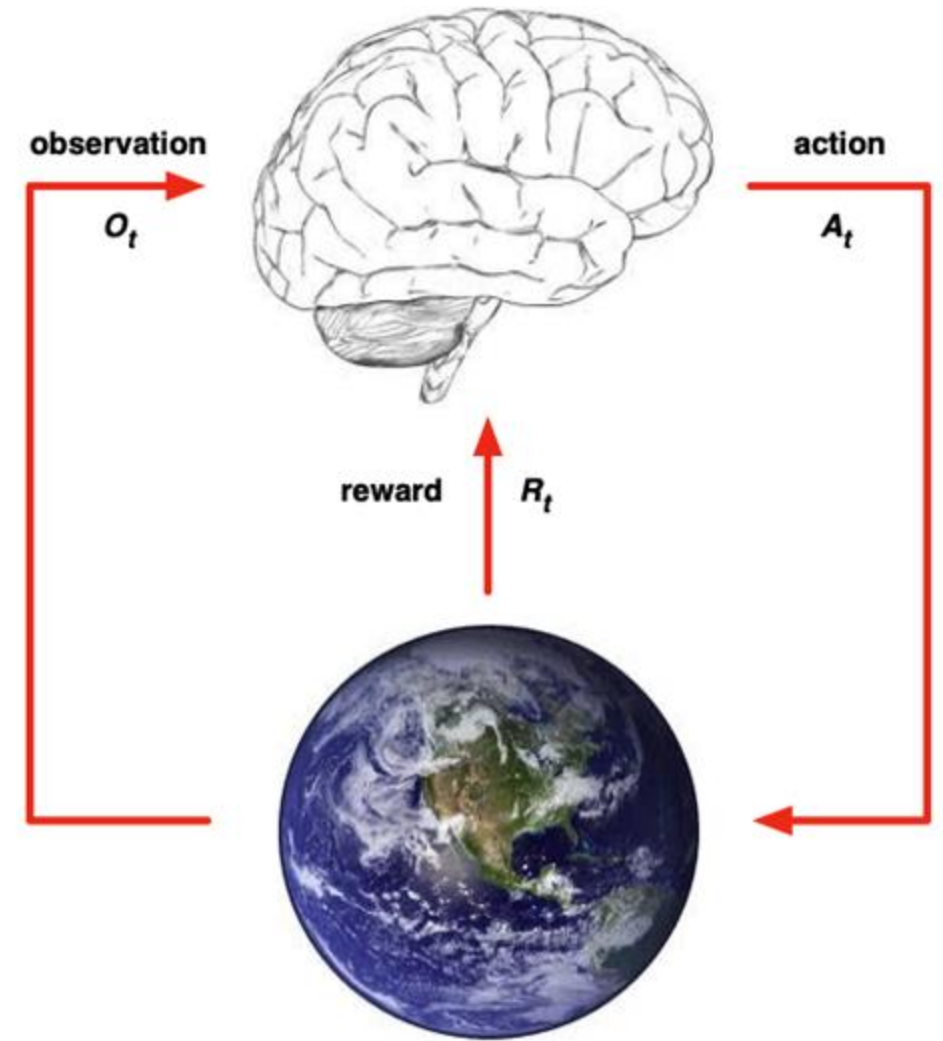
- Fly stunt manoeuvres in a helicopter
 - + reward for following desired trajectory
 - – reward for crashing
- Manage an investment portfolio
 - + reward for each \$ in bank
- Control a power station
 - + reward for producing power
 - – reward for exceeding safety thresholds
- Make a humanoid robot walk
 - + reward for forward motion
 - – reward for falling over
- Play many different Atari games better than humans
 - +/- reward for increasing/decreasing score

Sequential Decision Making

- **Goal: select actions to maximise total future reward**
 - Actions may have long term consequences
 - Reward may be delayed
 - It may be better to sacrifice immediate reward to gain more long-term reward
- Examples:
 - A financial investment (may take months to mature)
 - Refuelling a helicopter (might prevent a crash in several hours)
 - Blocking opponent moves (might help winning chances many moves from now)

Agent and Environment

- At each step t the agent:
 - Executes action A_t
 - Receives observation O_t
 - Receives scalar reward R_t
- The environment:
 - Receives action A_t
 - Emits observation O_{t+1}
 - Emits scalar reward R_{t+1}
- t increments at environment step



History and State

- The history is the sequence of observations, actions, rewards

$$H_t = O_1, R_1, A_1, \dots, A_{t-1}, O_t, R_t$$

- i.e. all observable variables up to time t
- What happens next depends on the history, including:
 - What actions the agent selects
 - What the agent observes from the environment
 - What rewards the environment gives
- **State is the information used to determine what happens next**
 - Includes agent state and environment state
 - Agent chooses action based on history
 - state is a function of the history:
 - $S_t = f(H_t)$

Information State

- An information state (a.k.a. Markov state) contains all useful information from the history
- A state S_t is Markov if and only if

$$p(S_{t+1} | S_t, A_t) = p(S_{t+1} | H_t, A_t)$$

- “The future is independent of the past given the present”
- Once the state is known, the history may be thrown away
 - i.e. The state is a sufficient statistic of the future

Markov Assumption

- Simple and often can be satisfied if include some history as part of the state
- In practice often assume most recent observation is sufficient statistic of history:

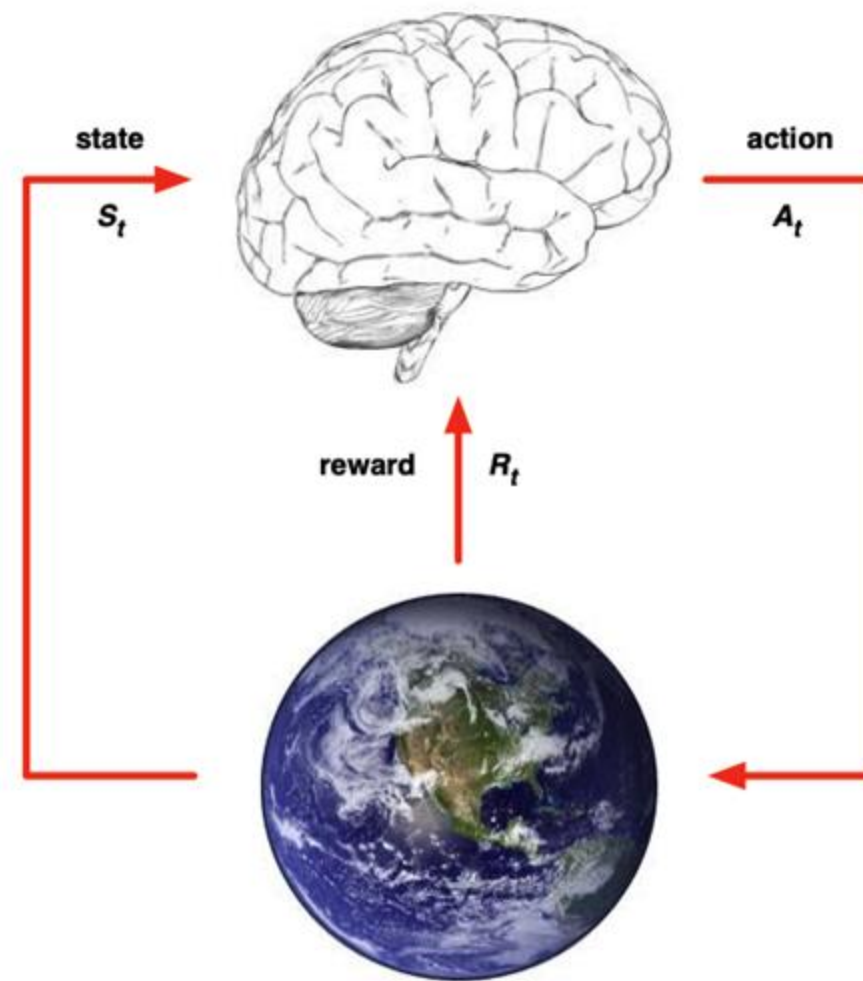
$$S_t = O_t$$

State t equals observation t

- State representation has big implications for:
 - Computational complexity
 - Data required
 - Resulting performance

Fully Observable Environments

- Full observability: agent directly observes environment state: $S_t = O_t$
- Agent state = environment state = information state
- Formally, this is a **Markov decision process (MDP)**



Partially Observable Environments

- Partial observability: agent indirectly observes environment:
 - A robot with camera vision isn't told its absolute location
 - A trading agent only observes current prices
 - A poker playing agent only observes public cards
- Now agent state does not equal to environment state
- Formally this is a **partially observable Markov decision process (POMDP)**
- Agent must construct its own state representation
 - Beliefs of the state!

Markov Decision Process

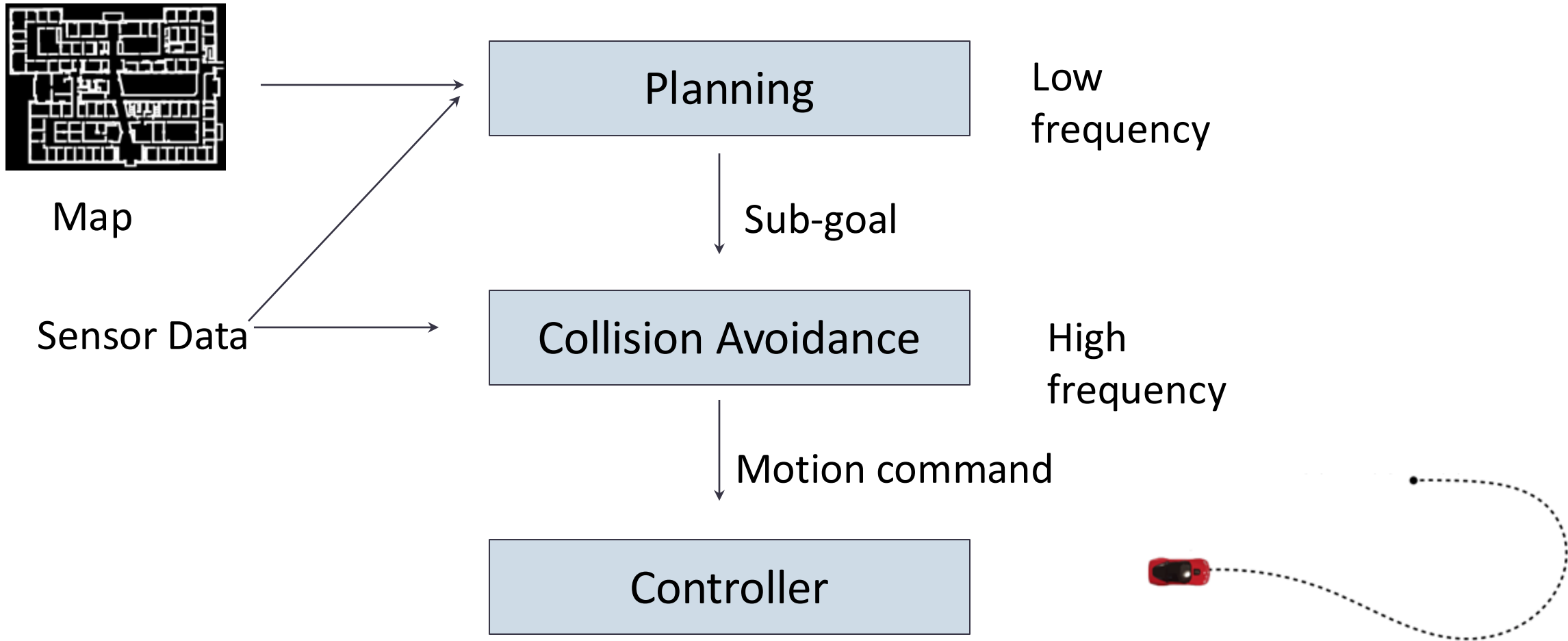
- Markov decision processes formally describe an environment for reinforcement learning
- Where the environment is fully observable
 - i.e. The current state completely characterises the process
- Almost all RL problems can be formalised as MDPs, e.g.
 - Optimal control primarily deals with continuous MDPs
 - Partially observable problems can be converted into MDPs
 - Bandits are MDPs with one state

MDPs in Robot Navigation

- Challenges in Robot Navigation
 - Calculate the **optimal path**
 - Generate appropriate actions in the case of unforeseen objects
 - Predicting the motion of and planning around moving obstacles
 - Take the **uncertainties** in the robot's action execution into account

MDPs in Robot Navigation

Classic Layered Architecture:



MDPs in Robot Navigation

Classic Layered Architecture:



Map

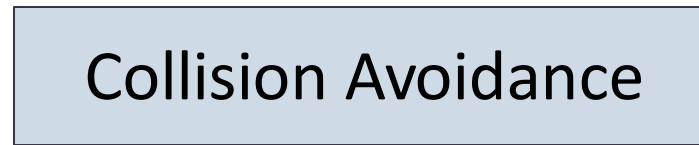
Sensor Data



Low
frequency

We do MDP at
the planning
level

Sub-goal



High
frequency

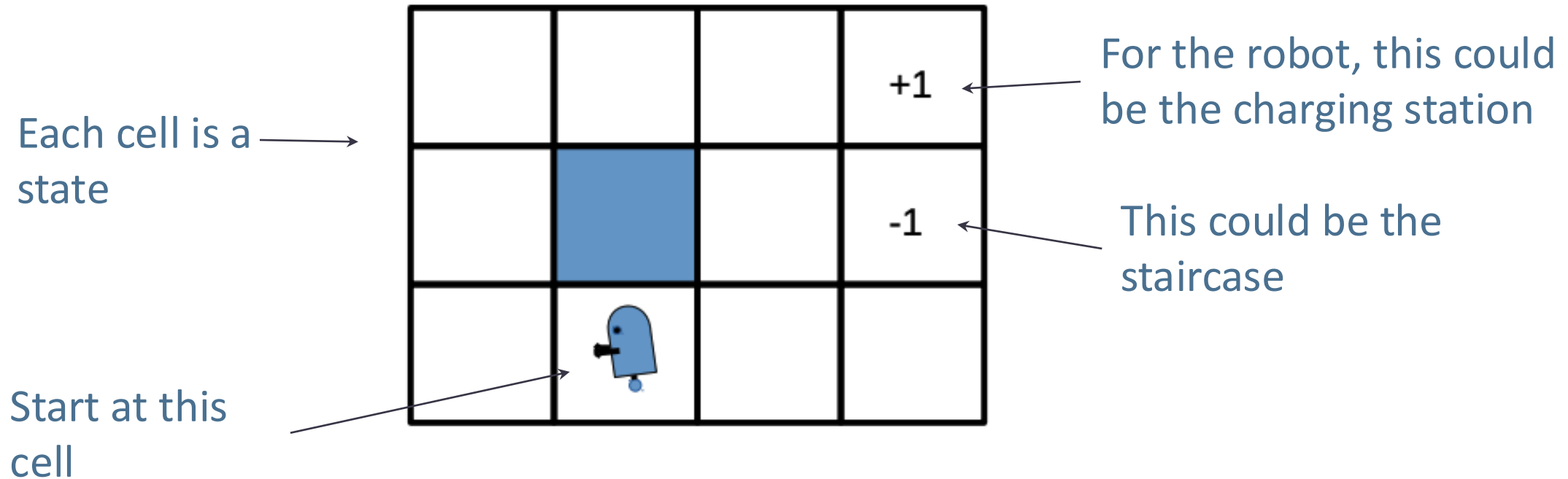
Motion command



Introduction

Example: Grid World

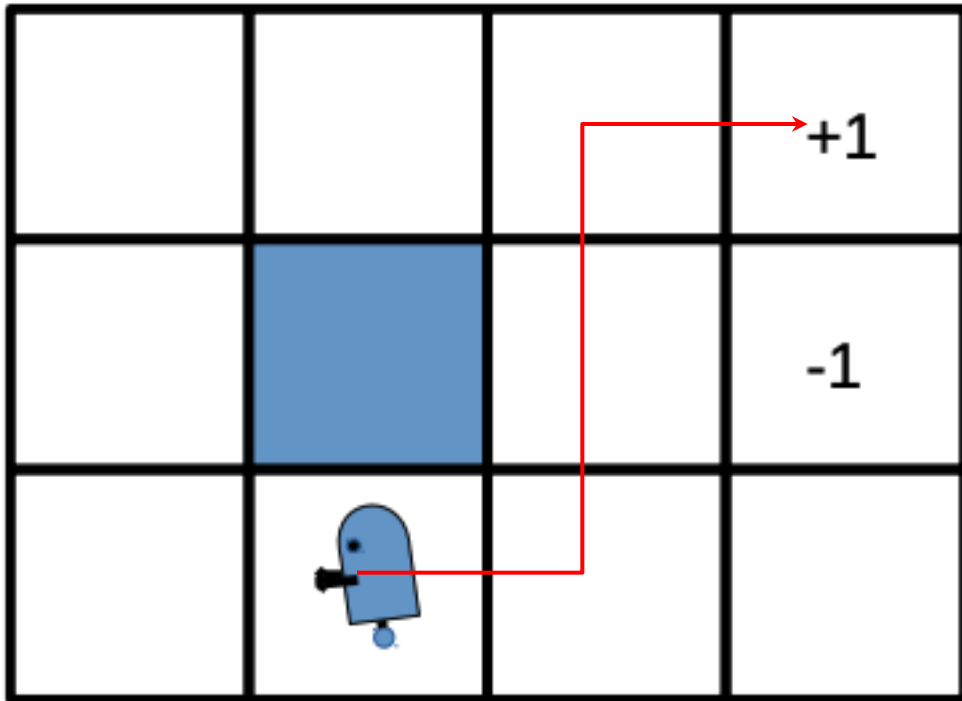
- A maze-like problem
 - The agent lives in a grid
 - Walls block the agent's path
- Agent acting in this environment
- Its mission is to reach the goal marked by +1 avoiding the cell labeled as -1



Introduction

Example: Grid World

- At a glance, this is easy to solve! Use a search algorithm, e.g., A*
- Best solution (shortest path) is the action sequence [Right, Up, Up, Right]

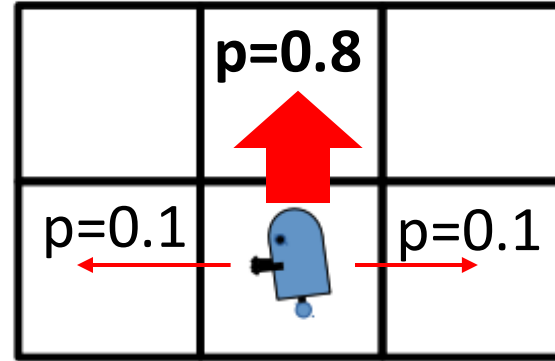


Why That Be Problematic?

- Non-perfect execution: actions are performed with a probability < 1
- What are the best actions for an agent under this constraint?

Non-Deterministic Transitions

- Consider non-deterministic (stochastic) transition model:

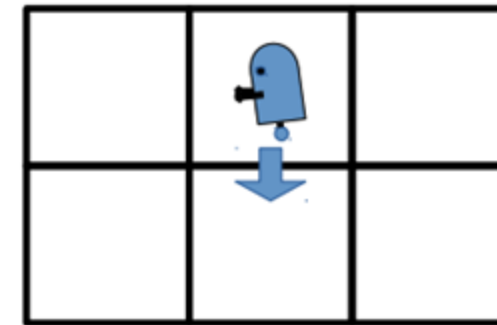
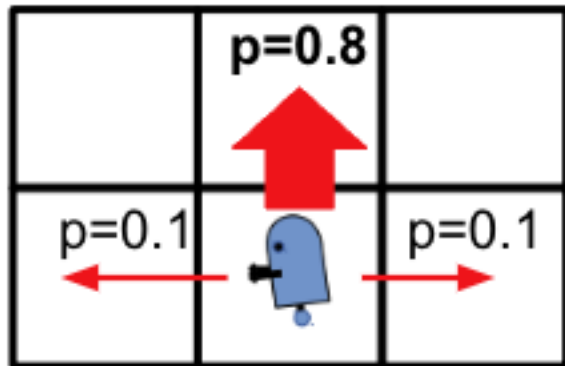


Desired action is not executed 100%. Let's say only 80%

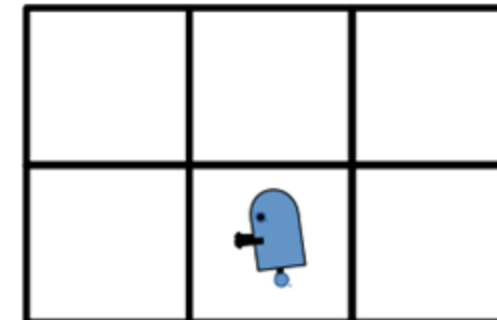
- Intended action is executed with $p=0.8$
- With $p=0.1$, the agent moves left or right
- Bumping into a wall “reflects” the robot
 - So it stays in the cell

Markov Decision Process (MDP)

- Deterministic
 - same action always has same outcome
- Stochastic / Non-deterministic
 - same action could have different outcomes

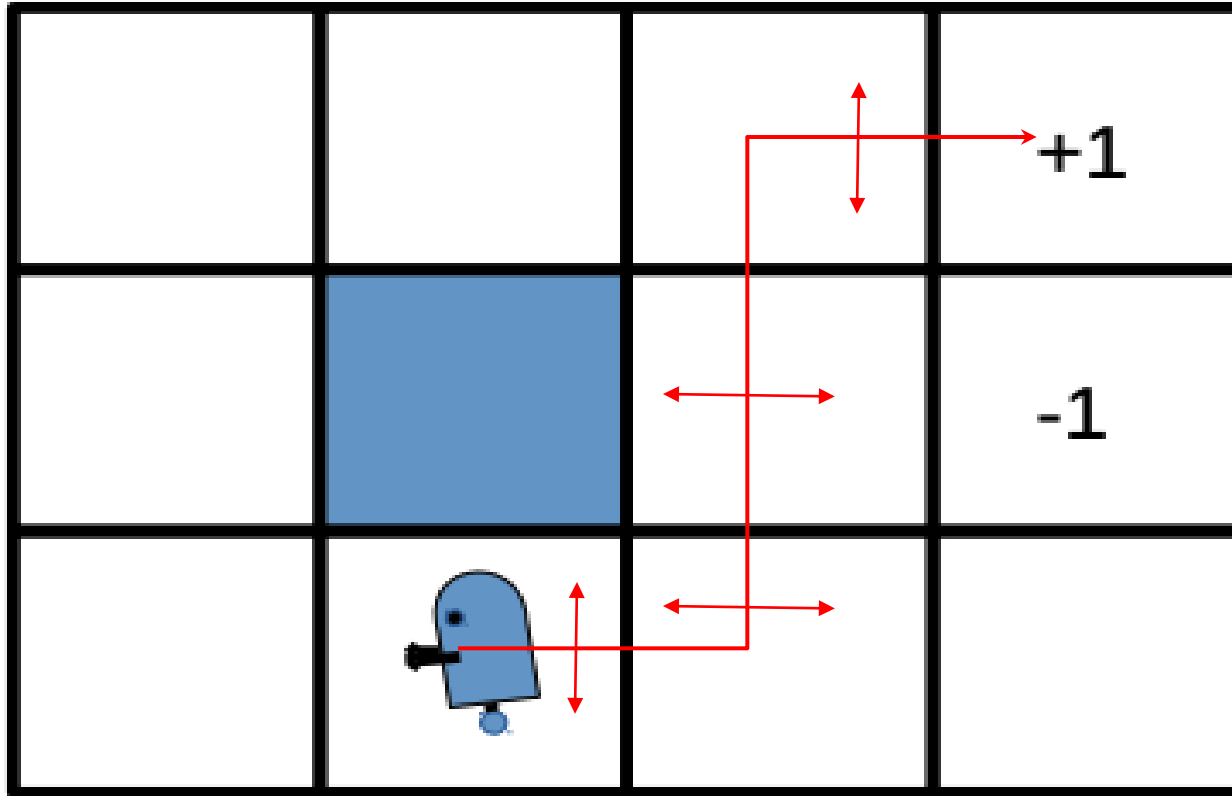


1.0



MDP Motivation

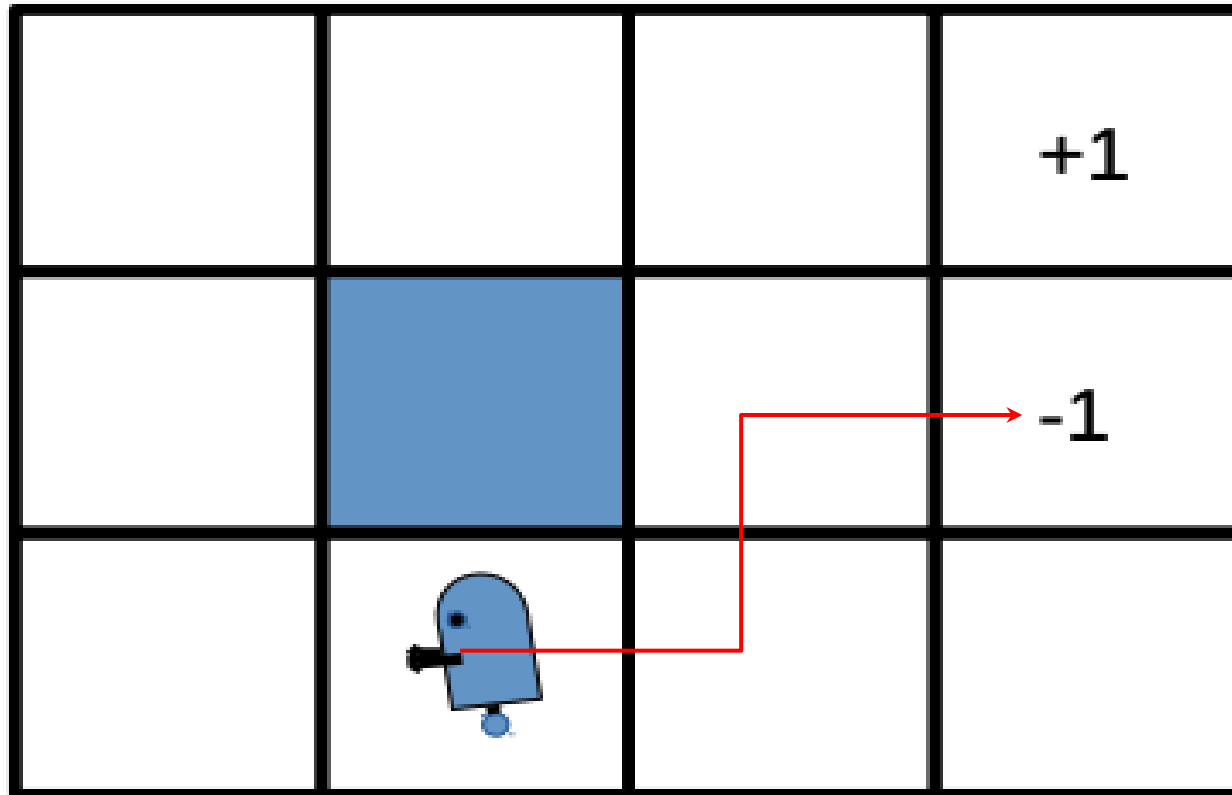
- Executing the A* plan...



Always a small chance
the robot will fail to
execute the right
command..

MDP Motivation

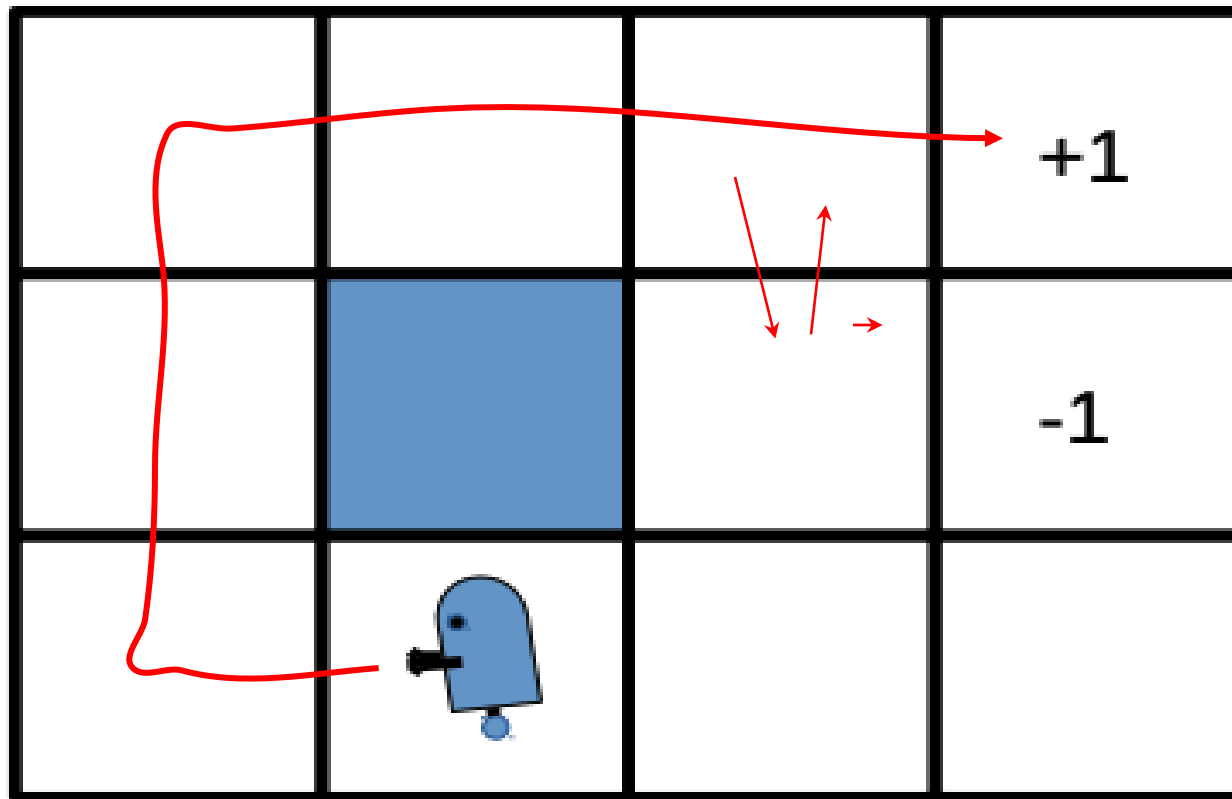
- Executing the A* plan...
 - This will happen sooner or later..



So the robot is dead now
How do we prevent this?

MDP Motivation

- Use longer path with lower probability to reach the bad outcome



So this route the robot is highly unlikely to die (it requires two bad execution, $10\% \times 10\% = 1\%$ chance)

Transition Model

- Probability to reach the next state s' from state s by choosing action a

State set: $s \in S$

Action Set: $a \in A$

$$T(s, a, s')$$

Sometimes people use $\mathcal{P}$ to represent state transition matrix

- Markov Property:
 - The transition probability from s to s' depend only on the current state s and not on the history of earlier states

Probabilistic models for State Estimation

- **State transition** model / dynamics model / **motion model**

State
transition
prob.

$$P(x_t | x_{t-1}, u_t)$$

Action at
time t

It is the same idea as in MDPs/RL
problems

Reward

- In each state s , the agent receives a reward $R(s)$
 - The reward may be positive or negative
- This can be generalized to be a reward function $R(s, a, s')$
 - Given state, action, next state
- Here in our grid world example: considering only $R(s)$

Reward

- In our example, the reward is **-0.04** in all states (e.g. the cost of motion) except the terminal states (reward **+1/-1**)
- A small negative reward at each state gives an **incentive to reach the goal quickly**
 - **This is usually called the slack reward**
- Or: “living in this environment is not enjoyable”

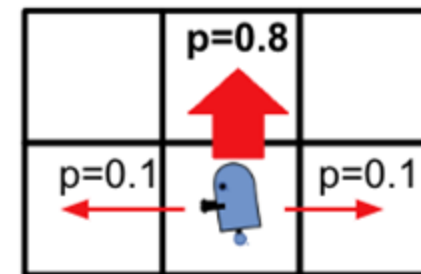
- 0.04	- 0.04	- 0.04	+1
- 0.04		- 0.04	-1
- 0.04	- 0.04	- 0.04	- 0.04

MDP Definition

- Given a sequential decision problem in a fully observable, stochastic environment with a known **Markovian transition model**
- Then a Markov Decision Process (MDP) is defined by the components
 - Set of states: S
 - Set of actions: A
 - Initial state: s_0
 - **Transition model:** $T(s, a, s')$
 - **Reward function:** $R(s) / R(s, a, s')$
 - An MDP is a 4-tuple: $M = (S, A, T, R)$

For reinforcement learning, the transition model and/or the reward function need to be learned

-0.04	-0.04	-0.04	+1
-0.04		-0.04	-1
-0.04	-0.04	-0.04	-0.04



MDP Definition

- **Transition model:** $T(s, a, s')$
 - This is a huge table!
 - **11 (cell) x 4(action) x 11 = 484 entries**
 - For now, we give this as input to the agent

$T(s_{11}, E, \dots)$

$T(s_{31}, \ddot{N}, s_{11}) = 0$

$T(s_{31}, \ddot{N}, s_{32}) = 0.8$

$T(s_{31}, \ddot{N}, s_{21}) = 0.1$

$T(s_{31}, \ddot{N}, s_{41}) = 0.1$

$\dots$

How do we maximize the reward?

Sometimes the transition model is expressed as the state transition model and the policy:

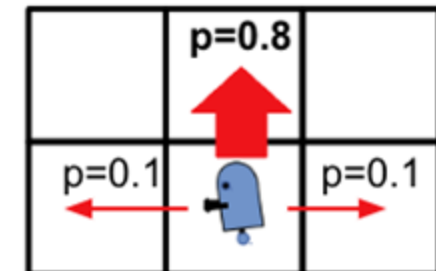
$$\mathcal{P}_{s,s'}^{\pi} = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{P}_{ss'}^a$$

Same goes for the Reward function: $R(s) / R(s, a, s')$

$$\mathcal{R}_s^{\pi} = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{R}_s^a$$

Policy: what action to take given state

-0.04	-0.04	-0.04	+1
-0.04		-0.04	-1
-0.04	-0.04	-0.04	-0.04



Policy

- An MDP solution is called **policy** π
- A policy is a mapping from states to actions

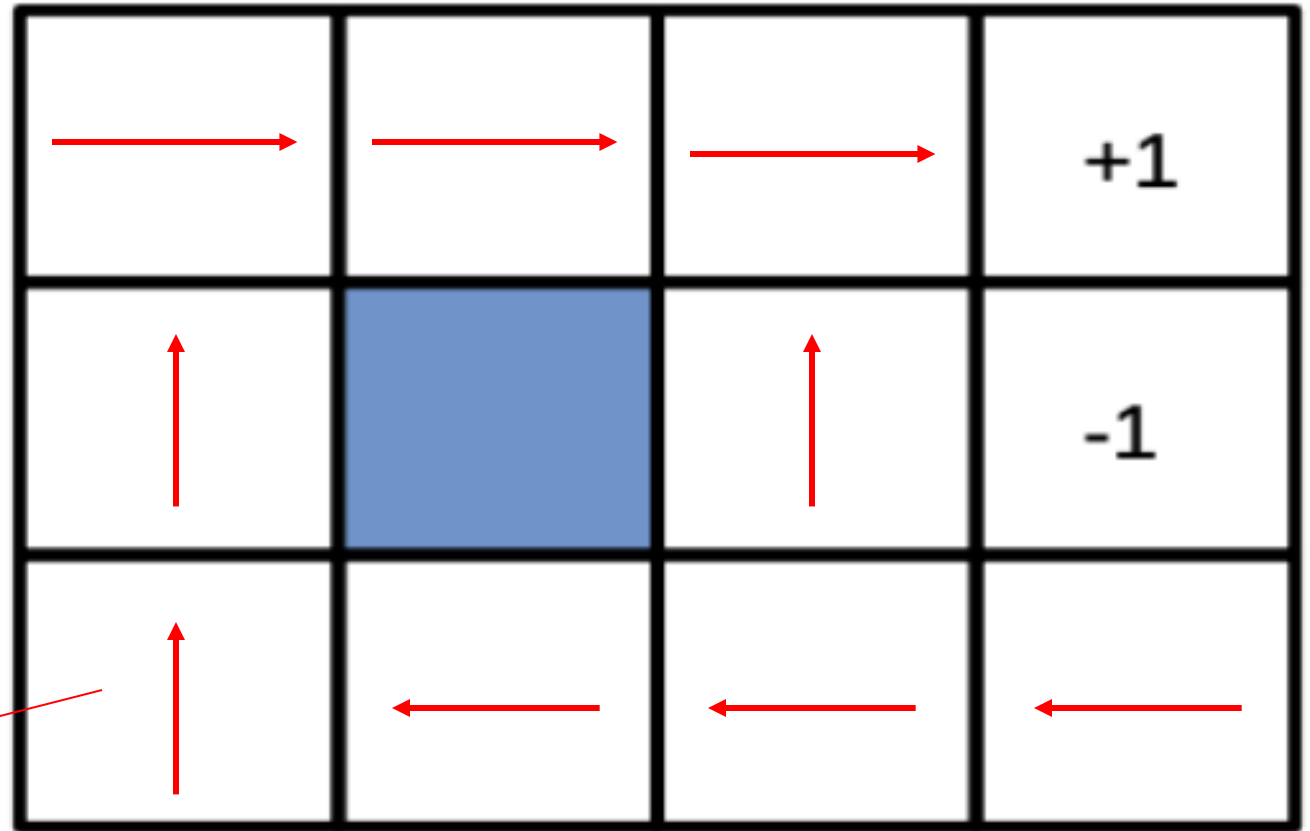
$$\text{policy} : \text{States} \mapsto \text{Actions}$$

- A policy tells the agent **what to do next**
 - **Givens and action for each state**
- Let $\pi(s)$ be the action that π specifies for s
- Among the many policies that solve an MDP, the optimal policy π^* is what we seek. We'll see later what optimal means

Policy

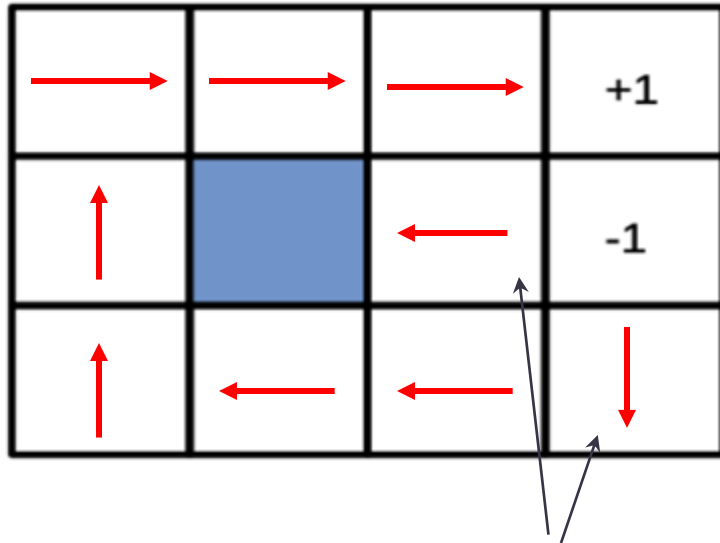
- The optimal policy for our example
 - **Conservative choice**
 - Take long way around as the cost per step of $R=-0.04$ is small compared with the penalty to fall down the stairs and receive a **-1** reward

This policy means: if the robot is in this cell (state), always take the up action

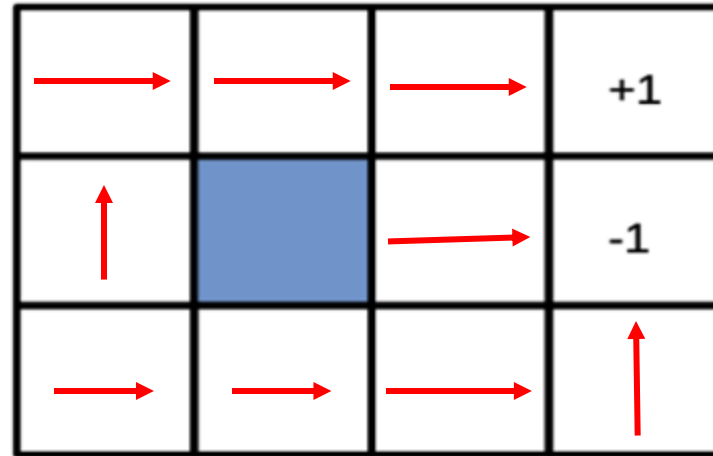


Policy

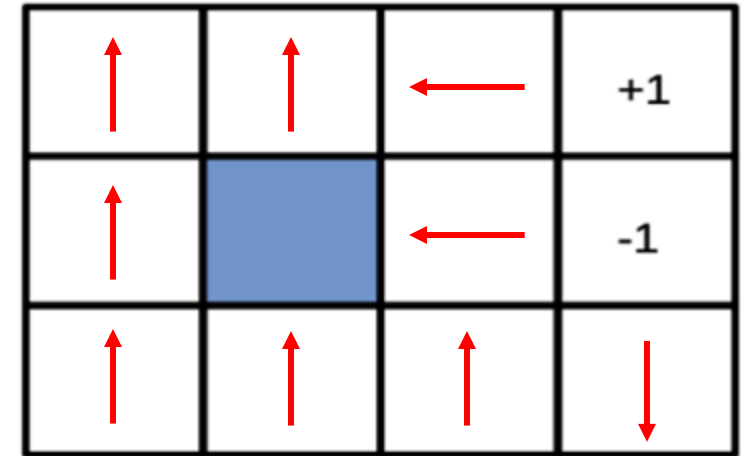
- When the balance of risk and reward changes, other policies are optimal
 - Originally $R = -0.04$ for all other states



If $R = -0.01$. Then no risks are taken in these two state.



If $R = -2$ Leave as soon as possible (staying is worse than falling off stairs)

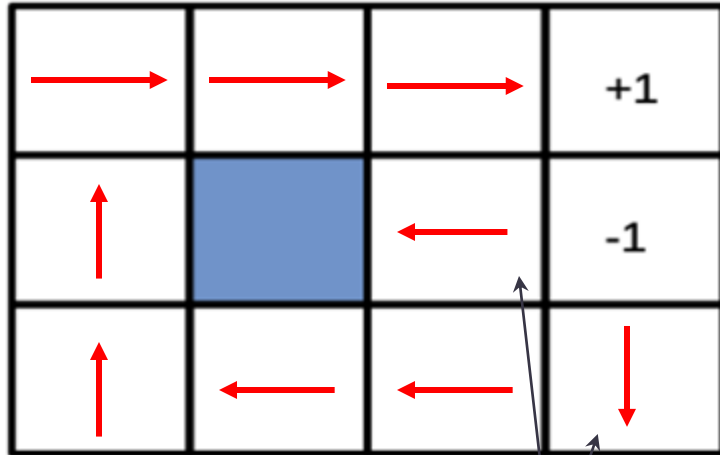


$R > 0$ Never leave. (The robot gets positive reward by staying)

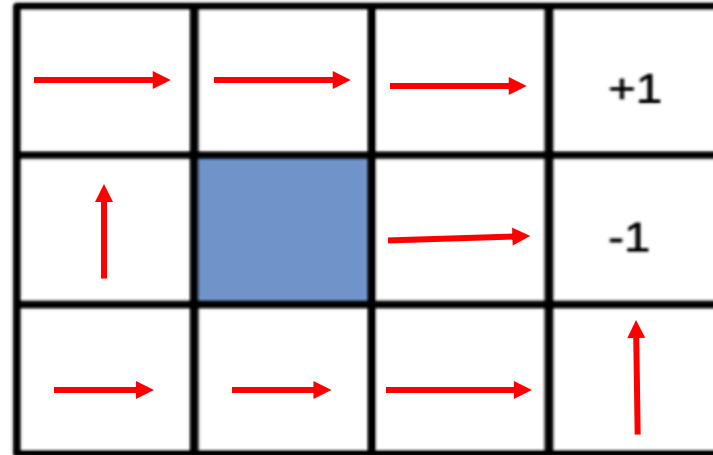
Policy

- When the balance of risk and reward changes, other policies are optimal
 - Originally $R = -0.04$ for all other states

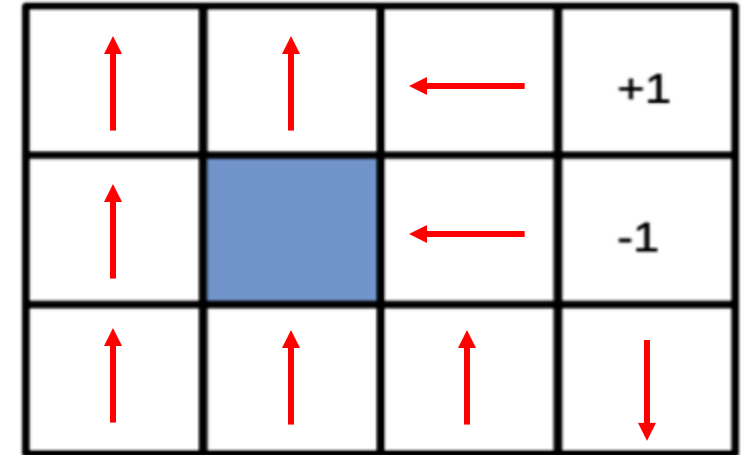
The rewards impact the policy!



$R = -0.01$ No risks are taken.



$R = -2$ Leave as soon as possible.
(staying is worse than falling off stairs)



$R > 0$ Never leave

Utility of a State

- The **utility of a state** $U(s)$ quantifies the **benefit** of a state for the overall task
- We first define $U^\pi(s)$ to be **expected utility of all state sequences that start from state s given π**

$$U^\pi(s) = E \left[\sum_{t=0}^{\infty} R(s_t) \mid \pi, s_0 = s \right]$$

- $U(s)$ evaluates (and encapsulates) all possible futures **from s onwards**
- **This is also called the state-value function and denoted by $V^\pi(s)$**
- Think of this as the expected return starting from state s given a policy
 - Long-term value of state s

Two Value Functions

- **State-value function (utility of state) / V function**
 - Expected return starting from state s , following policy

$$V^{\pi}(s) = U^{\pi}(s) = E \left[\sum_{t=0}^{\infty} R(s_t) \mid \pi, s_0 = s \right]$$

- **Action-value function / Q function / Q-value**
 - Expected return starting from state s , **taking action a** and then following policy

$$Q^{\pi}(s, a) = E \left[\sum_{t=0}^{\infty} R(s_t) \mid \pi, s_0 = s, a_0 = a \right]$$

Two Value Functions

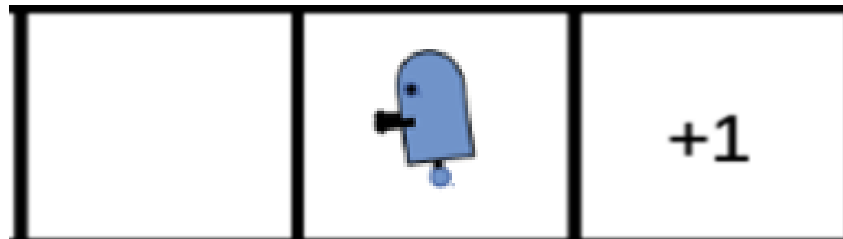
- State-value function (utility of state) / **V function**
- Action-value function / **Q function** / Q-value
- Conversion

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) Q^{\pi}(s, a)$$

- Why two functions?
 - Note: value functions always conditional on some policy
 - The Q-value lets you **play a hypothetical** of potentially taking a different action in the first time step
 - Example: compare the state value and Q value

Q function: Expected return starting from state s , **taking action a** and then following policy

So taking the sum over all actions gives you the state-value



Utility of a State / State-Value

- With this definition, we can express $U^\pi(s)$ as **a function of its next state s'**

$$V^\pi(s)$$

$$\begin{aligned} U^\pi(s) &= E \left[\sum_{t=0}^{\infty} R(s_t) \mid \pi, s_0 = s \right] \\ &= E \left[R(s_0) + R(s_1) + R(s_2) + \dots \mid \pi, s_0 = s \right] \\ &= E \left[R(s_0) \mid s_0 = s \right] + E \left[R(s_1) + R(s_2) + \dots \mid \pi \right] \\ &= R(s) + E \left[\sum_{t=0}^{\infty} R(s_t) \mid \pi, s_0 = s' \right] \\ &= R(s) + U^\pi(s') \end{aligned}$$

The cumulative reward from now equals the current reward plus cumulative rewards from the next state

Optimal Policy

- The **optimal policy** π^* in s chooses the action a that maximizes the expected utility of s (and of s')

$$\pi^*(s) = \operatorname{argmax}_a E[U^\pi(s)]$$

- Expectation **over all policies**

Optimal Policy

- Substituting $U^\pi(s)$

$$\pi^*(s) = \operatorname{argmax}_a E[U^\pi(s)]$$

Function of the next state

$$= \operatorname{argmax}_a E[R(s) + U^\pi(s')]$$

$$= \operatorname{argmax}_a E[R(s)] + E[U^\pi(s')]$$

$$= \operatorname{argmax}_a E[U(s')]$$

This is constant (actions only impacts future rewards)

$$= \operatorname{argmax}_a \sum_{s'} T(s, a, s') U(s')$$

Recall that $E[X]$ is the weighted average of all possible values that X can take on

Optimal policy results in the maximum product of all possible future state utilities and transition probabilities

True Utility of a State

- The **true utility of a state** $U(s)$ is then obtained by application of the **optimal policy**, i.e. $U^{\pi^*}(s) = U(s)$
- We find

$$\begin{aligned} U(s) &= \max_a E [U^{\pi}(s)] \\ &= \max_a E [R(s) + U^{\pi}(s')] \\ &= \max_a E [R(s)] + E [U^{\pi}(s')] \\ &= R(s) + \max_a E [U(s')] \quad \text{Future utilities} \\ &= R(s) + \max_a \sum_{s'} T(s, a, s') U(s') \end{aligned}$$

Utility of a State

Bellman Equation

$$U(s) = R(s) + \max_a \sum_{s'} T(s, a, s') U(s')$$

- This is the Bellman equation: Direct relationship between the **utility of a state** and the **utility of its neighbors**
- Utility of a state = the immediate reward + expected utility of the next state, given we choose the **optimal** actions
- We can do this computation for all states

Bellman Equation

$$U(s) = R(s) + \max_a \sum_{s'} T(s, a, s') U(s')$$

- For each state there is a Bellman equation to compute its utility
- There are **n states** and **n unknowns**
- Can we solve the system using linear algebra?
- No! The max-operator that chooses the optimal action makes the system non-linear
- Go for an **iterative approach (dynamic programming)**

Discounting

- The utility of a state sequence is often defined as the sum of discounted rewards

$$U^{\pi}(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t) \mid \pi, s_0 = s \right]$$

- The discount factor: $\gamma \in [0, 1]$
- Discounting says that **future** rewards are **less significant** than **current** rewards. This is a natural model for many domains
- The other expressions change accordingly

Bellman Equation for Value Functions

- Optimal solution for both value functions can be derived using Bellman Equation
 - We can derive optimal policy from optimal value function by maximising over them
 - An MDP is “solved” when we know the optimal value function

$$U^{\pi^*}(s) = R(s) + \gamma \max_a \sum_{s'} T(s, a, s') U^{\pi^*}(s')$$

$$Q^{\pi^*}(s, a) = R(s) + \gamma \max_a \sum_{s'} T(s, a, s') Q^{\pi^*}(s', a')$$

Solving the Bellman Optimality Equation

- Bellman Optimality Equation is non-linear (the max operator)
- Many iterative solution methods
 - Value Iteration
 - Policy Iteration
 - Q-learning
 - SARSA (State–action–reward–state–action)
 - ...

Utility of a State

Example:

- **State utilities** for our example
- Utilities are higher closer to the goal as fewer steps are needed to reach it

$$U^{\pi}(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t) \mid \pi, s_0 = s \right]$$

0.81 2	0.86 8	0.91 8	+1
0.76 2		0.66	-1
0.70 5	0.65 5	0.61 1	0.38 8

Iterative Computation for Value Function

- The utility is computed iteratively:

$$U_{i+1}(s) \leftarrow R(s) + \max_a \sum_{s'} T(s, a, s') U_i(s')$$

$$U^* = \lim_{t \rightarrow \infty} U_t$$

- Optimal utility:
- Abort, if change in utility is below a threshold

Algorithm 1: Value Iteration

In: An MDP with

- States and action sets S, A ,
- Transition model $T(s, a, s')$,
- Reward function $R(s)$,
- Discount factor γ

Out: The utility of all states U

$U' \leftarrow 0$

repeat

$U \leftarrow U'$

 foreach state s in S do

$U(s) \leftarrow R(s) + \gamma \max_a \sum_{s'} T(s, a, s') U(s')$

 end

until *close-enough*(U, U')

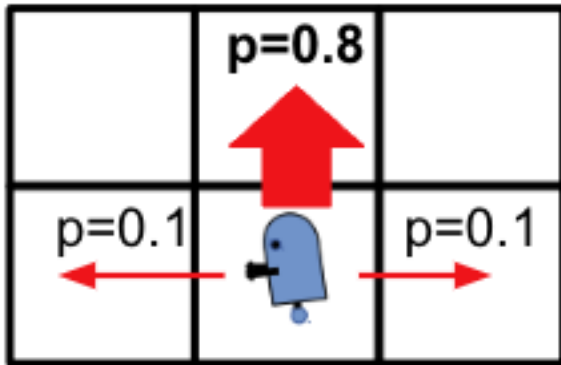
return U

Value Iteration

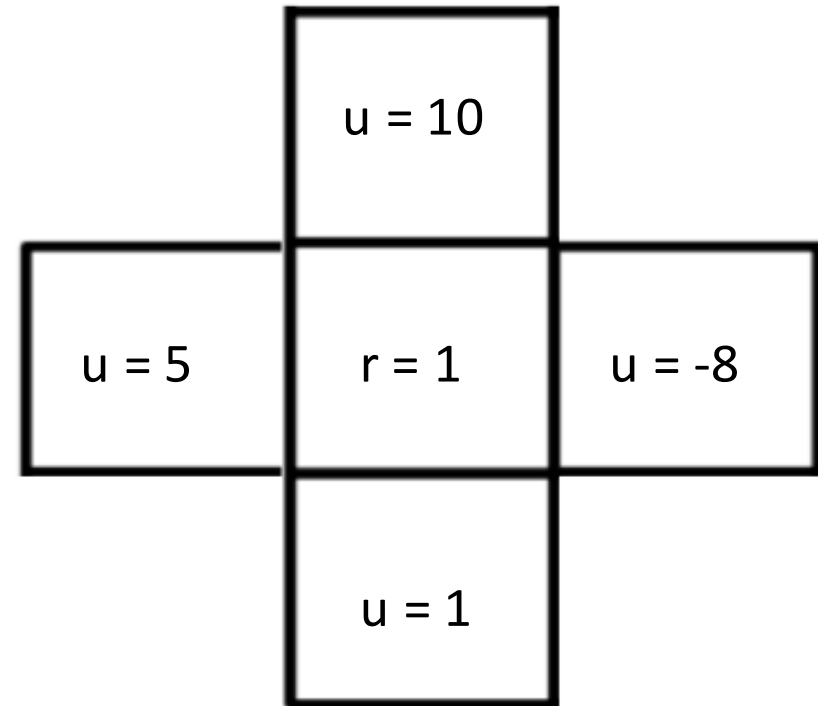
Example:

- Calculate utility of the center cell

$$U_{i+1}(s) \leftarrow R(s) + \max_a \sum_{s'} T(s, a, s') U_i(s')$$



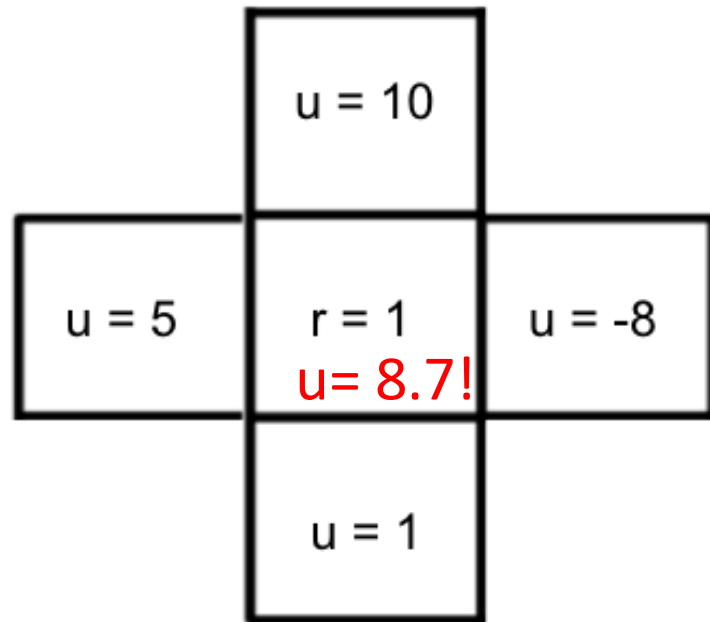
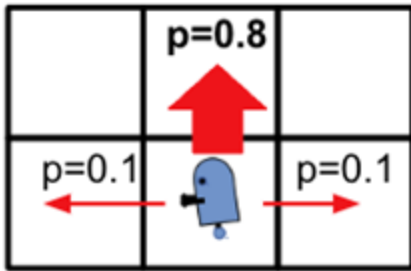
Transition Model



State space (u= utility, r= reward)

Value Iteration

$$U_{i+1}(s) \leftarrow R(s) + \max_a \sum_{s'} T(s, a, s') U_i(s')$$



= reward + max{
 $0.1 \cdot 1 + 0.8 \cdot 5 + 0.1 \cdot 10$, (Left)
 $0.1 \cdot 5 + 0.8 \cdot 10 + 0.1 \cdot -8$, (Up)
 $0.1 \cdot 10 + 0.8 \cdot -8 + 0.1 \cdot 1$, (Right)
 $0.1 \cdot -8 + 0.8 \cdot 1 + 0.1 \cdot 5$, (Down)}

= $1 + \max\{5.1, 7.7, -5.3, 0.5\}$

= $1 + 7.7$

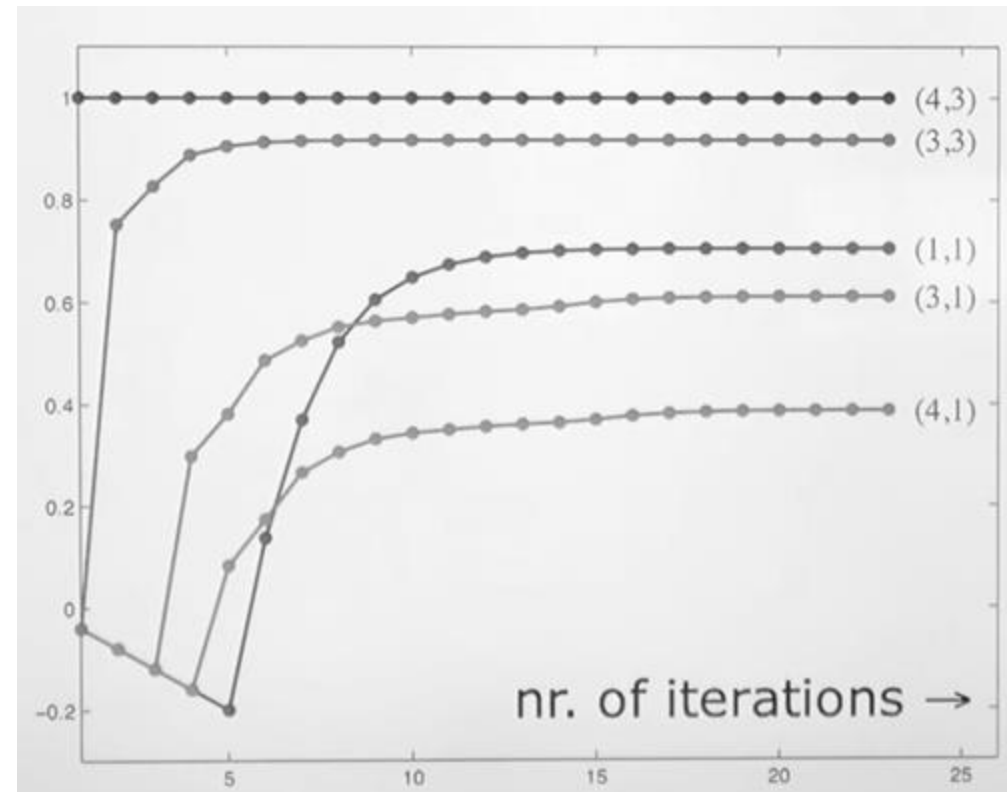
= 8.7

Value Iteration

0.812	0.868	0.918	+1
0.762		0.66	-1
0.705	0.655	0.611	0.388

(1, 1)

(4, 3)



Value Iteration Properties

- Value iteration finds the **optimal solution** to the Markov Decision Problem
- Converges to the **unique solution** of Bellman equation system
- Initial values for U' are arbitrary
- Value iteration propagates information through the state space by means of local updates

Optimal Policy

- How to compute the **optimal policy**?
- Can be easily extracted by

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} T(s, a, s') U(s')$$

The optimal policy is implied by the optimal value function

- Note
 - $U(s)$ and $R(s)$ are different quantities
 - $R(s)$ = **short-term** reward for being in s
 - $U(s)$ = **long-term** reward **from s onwards**

Problems with Value Iteration

- Problem 1: It's slow – $O(S^2A)$ per iteration
- Problem 2: The “max” at each state rarely changes
- Problem 3: The policy often converges long before the values

Policy Iteration:

- Alternative approach for optimal values:
 - Step 1: **Policy evaluation**: calculate utilities for some fixed policy (not optimal utilities!) until convergence
 - Step 2: **Policy improvement**: update policy using one-step look-ahead with resulting converged (but not optimal!) utilities as future values
 - Repeat steps until policy converges
- Can converge (much) faster under some conditions

Comparison

- Both value iteration and policy iteration compute the same thing (all optimal values)
- In value iteration:
 - Every iteration updates both the values and (implicitly) the policy
 - We don't track the policy, but taking the max over actions implicitly recomputes it
- In policy iteration:
 - We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
 - After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
 - The new policy will be better (or we're done)
- Both are dynamic programs for solving MDPs

Summary

- Planning becomes more complicated under uncertainty
- Uncertainty in action execution leads to a Markov Decision Processes (MDP)
- Value/Policy iteration as a key algorithm for solving an MDP
 - Using Bellman equation
 - Step 1: take correct first action
 - Step 2: keep being optimal
- Optimal policy can be extracted from optimal values



Thanks for your attention!

Changhao Chen
HKUST (GZ)

changhaochen@hkust-gz.edu.cn

Homepage: [changhao-chen@github.io](https://github.com/changhao-chen)